

10x Senior Engineer Interview Preparation Guide

Table of Contents

1. [JavaScript & TypeScript Fundamentals](#)
 2. [Node.js Concepts](#)
 3. [SQL and Database Management](#)
 4. [ORM Technology](#)
 5. [API Security and Scalability](#)
 6. [NestJS and Express.js](#)
 7. [Design and Architecture](#)
 8. [Performance and Optimization](#)
 9. [Git and Version Control](#)
 10. [Algorithm and Data Structure Problems](#)
 11. [Authentication and Authorization](#)
 12. [Design Patterns and SOLID Principles](#)
 13. [System Design](#)
 14. [Code Analysis Questions](#)
-

JavaScript & TypeScript Fundamentals

Q: Explain scoping and hoisting in JavaScript.

A: In JavaScript, scoping determines the visibility and accessibility of variables, while hoisting is a behavior where variable and function declarations are moved to the top of their containing scope

during compilation.

Scoping in JavaScript:

- **Global Scope:** Variables declared outside any function or block are accessible everywhere
- **Function Scope:** Variables declared with `var` inside a function are only accessible within that function
- **Block Scope:** Variables declared with `let` and `const` are limited to the block they're defined in (introduced in ES6)

Hoisting:

- Function declarations are completely hoisted with their body
- Variable declarations using `var` are hoisted but initialized with `undefined`
- `let` and `const` are hoisted but remain in the "temporal dead zone" until their declaration is reached, throwing an error if accessed before
- Class declarations are hoisted but not initialized

This behavior explains code like:

javascript

```
console.log(x); // undefined, not an error  
var x = 5;
```

```
sayHello(); // Works because function declarations are fully hoisted  
function sayHello() {  
  console.log("Hello");  
}
```

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization  
let y = 10;
```

As a senior engineer, I ensure our team avoids hoisting-related bugs by declaring variables at the top of their scope and using `let`/`const` over `var` to maintain clearer, more predictable code behavior.

Q: Explain Promises, Callbacks, and Async/Await in JavaScript.

A: These are patterns for handling asynchronous operations in JavaScript, with each representing an evolution in making async code more manageable.

Callbacks:

The original pattern for handling async operations, where a function is passed as an argument to be executed when an operation completes:

javascript

```
fetchData(url, function(error, data) {  
  if (error) {  
    handleError(error);  
  } else {  
    processData(data);  
  }  
});
```

While functional, callbacks can lead to deeply nested structures ("callback hell") when chaining multiple async operations.

Promises:

Introduced to solve callback hell, promises are objects representing the eventual completion (or failure) of an async operation:

javascript

```
fetchData(url)  
  .then(data => processData(data))  
  .catch(error => handleError(error));
```

Promises enable chaining with `.then()` and centralized error handling with `.catch()`, creating more readable code.

Async/Await:

Built on promises, this syntax provides the most readable way to work with asynchronous code by making it look synchronous:

javascript

```
async function getData() {  
  try {  
    const data = await fetchData(url);  
    return processData(data);  
  } catch (error) {  
    handleError(error);  
  }  
}
```

In my experience leading backend teams, I've established patterns where we use async/await for most application code while understanding that promises power it under the hood. For library code or performance-critical paths, we sometimes use raw promises for finer control. I've also implemented custom promise utility functions for our specific business requirements, like retry mechanisms with exponential backoff for external API calls.

Q: What's the difference between `any` and `unknown` in TypeScript?

A: These types represent different approaches to type safety in TypeScript, with `unknown` being the type-safe counterpart to `any`.

`any` type:

- Bypasses type checking completely
- Variables of type `any` can have any properties accessed, called as functions, or assigned to any other type
- Effectively reverts TypeScript back to JavaScript's dynamic typing
- Propagates through your system, potentially causing uncaught type errors

`unknown` type:

- Represents values of unknown type, requiring type checking before use
- Cannot access properties, call methods, or assign to specific types without first performing type narrowing
- Forces explicit type checks or assertions before use
- Doesn't allow type-unsafe operations

Example of differences:

typescript

// With any

```
let dataAny: any = JSON.parse('{ "x": 10 }');  
dataAny.foo.bar; // No error during compilation, might fail at runtime
```

// With unknown

```
let dataUnknown: unknown = JSON.parse('{ "x": 10 }');  
dataUnknown.x; // Error: Object is of type 'unknown'
```

// Type narrowing required

```
if (typeof dataUnknown === 'object' && dataUnknown !== null && 'x' in dataUnknown) {  
  console.log((dataUnknown as { x: number }).x); // Safe access  
}
```

As a senior engineer who values type safety, I advocate for using `unknown` over `any` when dealing with values of uncertain types (like API responses or parsed JSON). We've implemented a policy in our projects where `any` usage requires explicit justification in code comments and is caught by our linting rules. This approach has significantly reduced runtime type errors in our production systems.

Q: Explain the difference between `call`, `apply`, and `bind` in JavaScript.

A: These are methods used to control the `this` context in JavaScript functions, but they differ in how arguments are passed and when the function is executed.

`call`:

- Executes the function immediately
- First argument sets the `this` value
- Remaining arguments are passed individually to the function

javascript

```
function greet(greeting) {  
  return `${greeting}, ${this.name}`;  
}  
  
const user = { name: 'Alice' };  
greet.call(user, 'Hello'); // "Hello, Alice"
```

`apply`:

- Executes the function immediately
- First argument sets the `this` value
- Second argument is an array containing all arguments for the function

javascript

```
greet.apply(user, ['Hello']); // "Hello, Alice"
```

`bind`:

- Returns a new function with `this` bound to the specified value
- Does not execute the function immediately
- Arguments can be partially applied

javascript

```
const greetUser = greet.bind(user);  
greetUser('Hello'); // "Hello, Alice"
```

```
// With partial application  
const sayHelloToUser = greet.bind(user, 'Hello');  
sayHelloToUser(); // "Hello, Alice"
```

In production systems, I've used these methods for:

- Implementing event handlers that need specific context
- Creating utility functions with consistent context across different execution environments
- Designing plugin systems where external code needs to run within our application's context
- Partial application for creating specialized functions from more general ones

The choice between them depends on the specific use case—`bind` for creating reusable functions with fixed context, and `call`/`apply` for immediate execution with dynamic context.

Q: What are generic functions in TypeScript and how do you use them?

A: Generic functions in TypeScript allow us to create reusable components that work with a variety of types while maintaining type safety. They enable us to define functions where the types of inputs relate to the types of outputs without specifying the exact type upfront.

Key concepts:

- Generic type parameters are defined using angle brackets: `<T>`
- Type parameters create relationships between parameters and return values
- They preserve type information throughout the function

Basic example:

typescript

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

// Type inference

```
const str = identity("hello"); // str is string  
const num = identity(42);      // num is number
```

// Explicit type arguments

```
const arr = identity<number[]>([1, 2, 3]); // arr is number[]
```

Real-world examples:

typescript

```
// Generic function with constraints
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

// Generic async data fetcher
async function fetchData<T>(url: string): Promise<T> {
  const response = await fetch(url);
  return response.json() as Promise<T>;
}

// Usage with interface
interface ApiResponse<T> {
  data: T;
  status: number;
}

async function fetchUser<T>(id: string): Promise<ApiResponse<T>> {
  // Implementation
}
```

In our production environment, I've implemented generic utility functions for data transformation pipelines that preserve type safety across multiple processing steps. We've also created generic repository patterns for database access that maintain strong typing from database to UI layer. This approach has significantly reduced type-related bugs and improved developer experience by providing better IDE autocompletion.

Node.js Concepts

Q: Explain the Node.js Event Loop.

A: The Node.js Event Loop is the core mechanism that enables Node's non-blocking, asynchronous I/O model, allowing it to handle thousands of concurrent connections with a single thread.

Key phases of the Event Loop:

1. **Timers:** Executes callbacks scheduled by `setTimeout()` and `setInterval()`
2. **Pending Callbacks:** Executes I/O callbacks deferred to the next loop iteration
3. **Idle, Prepare:** Used internally by Node.js
4. **Poll:** Retrieves new I/O events and executes their callbacks (or waits if needed)
5. **Check:** Executes callbacks registered via `setImmediate()`
6. **Close Callbacks:** Executes close event callbacks (e.g., `socket.on('close', ...)`)

Execution priority:

1. Synchronous code executes immediately
2. Microtasks queue (Promise callbacks, `process.nextTick()`) runs after each phase
3. Macrotasks (timers, I/O, `setImmediate()`) run according to their respective phases

Example showing execution order:

javascript

```
console.log('Start'); // 1

setTimeout(() => {
  console.log('Timeout'); // 5
}, 0);

Promise.resolve().then(() => {
  console.log('Promise'); // 3
});

process.nextTick(() => {
  console.log('NextTick'); // 2
});

setImmediate(() => {
  console.log('Immediate'); // 6
});

console.log('End'); // 4
```

Understanding the event loop is critical for building high-performance Node applications. In my experience architecting enterprise-scale Node.js services, I've used this knowledge to:

- Avoid CPU-intensive operations in the main thread by offloading them to worker threads
- Optimize promise chains to prevent starving the event loop
- Implement custom queuing mechanisms for rate-limiting and controlling concurrency
- Profile and diagnose event loop lag using metrics like event loop utilization (ELU)

This detailed understanding ensures our services remain responsive even under heavy load conditions.

Q: How would you handle errors in Node.js applications?

A: Error handling in Node.js requires a comprehensive strategy across synchronous and asynchronous code paths. Here's my approach as a senior engineer:

1. Error types and categorization:

- **Operational errors:** Expected errors that should be handled (network failures, validation errors)
- **Programmer errors:** Bugs that should crash and be fixed (undefined is not a function)
- **System errors:** Issues with the environment (out of memory)

2. Synchronous error handling:

- Use try/catch blocks for synchronous code paths
- Validate inputs early to prevent cascading errors

3. Asynchronous error handling:

- With callbacks: Follow the Node.js convention `(err, result) => {}`
- With promises: Use `.catch()` or try/catch with async/await
- Avoid unhandled promise rejections

4. Error propagation patterns:

javascript

// Middleware pattern for Express

```
const asyncHandler = (fn) => (req, res, next) => {  
  Promise.resolve(fn(req, res, next)).catch(next);  
};  
  
app.get('/users', asyncHandler(async (req, res) => {  
  const users = await userService.getAll();  
  res.json(users);  
}));
```

5. Centralized error handling:

javascript

```
// Express error handler
app.use((err, req, res, next) => {
  const statusCode = err.statusCode || 500;

  // Log detailed error for internal use
  logger.error({
    message: err.message,
    stack: err.stack,
    requestId: req.id,
    path: req.path
  });

  // Return sanitized error to client
  res.status(statusCode).json({
    error: statusCode < 500 ? err.message : 'Internal server error',
    requestId: req.id
  });
});
```

6. Domain-specific error classes:

javascript

```
class ApplicationError extends Error {
  constructor(message, statusCode, code) {
    super(message);
    this.statusCode = statusCode;
    this.code = code;
    this.name = this.constructor.name;
  }
}

class ValidationError extends ApplicationError {
  constructor(message, validationErrors) {
    super(message, 400, 'VALIDATION_ERROR');
    this.validationErrors = validationErrors;
  }
}
```

I've implemented comprehensive error handling systems for large-scale distributed applications that include:

- Integration with observability platforms like Datadog or New Relic
- Context preservation across asynchronous boundaries
- Automatic retry mechanisms with exponential backoff for transient failures
- Circuit breakers for external service dependencies

This approach has significantly improved our system resilience and reduced mean time to resolution (MTTR) for production incidents.

Q: What are streams and pipes in Node.js and when would you use them?

A: Streams and pipes are powerful Node.js abstractions for processing data in chunks rather than loading entire datasets into memory, enabling efficient handling of large files and real-time data.

Types of Streams:

1. **Readable streams:** Sources of data (file reads, HTTP requests)
2. **Writable streams:** Destinations for data (file writes, HTTP responses)
3. **Duplex streams:** Both readable and writable (TCP sockets)
4. **Transform streams:** Modify data as it's read and written (compression/encryption)

Key stream benefits:

- **Memory efficiency:** Process data in small chunks instead of loading everything at once
- **Time efficiency:** Start processing data as soon as it's available
- **Composability:** Chain multiple processing steps together with pipes

Example: Processing a large file

javascript

```
// Without streams - loads entire file into memory
fs.readFile('bigFile.csv', (err, data) => {
  // Process entire file at once
});

// With streams - processes file in chunks
const readStream = fs.createReadStream('bigFile.csv');
const transformStream = new Transform({
  transform(chunk, encoding, callback) {
    // Process chunk
    const transformed = processChunk(chunk);
    callback(null, transformed);
  }
});
const writeStream = fs.createWriteStream('output.csv');

readStream
  .pipe(transformStream)
  .pipe(writeStream)
  .on('finish', () => console.log('Processing complete'))
  .on('error', (err) => console.error('Error:', err));
```

Real-world applications where I've implemented streams:

1. **ETL pipelines:** Processing multi-gigabyte data files for business intelligence
2. **Real-time log processing:** Analyzing application logs on-the-fly
3. **Media processing:** Handling video transcoding without excessive memory usage
4. **API response streaming:** Sending large datasets to clients incrementally

I've also built custom stream implementations for specific business needs, like rate-limited API request streams and specialized transform streams for complex data normalization. In one project, we reduced memory usage by 70% and processing time by 40% by switching from a buffer-based approach to streams for a data migration service.

SQL and Database Management

Q: How would you find duplicates in a database table using SQL?

A: Finding duplicates in a database table is a common task that requires understanding both the business definition of a "duplicate" and SQL's grouping capabilities.

Basic approach using GROUP BY and HAVING:

```
sql

SELECT column1, column2, ..., COUNT(*) as count
FROM table_name
GROUP BY column1, column2, ...
HAVING COUNT(*) > 1;
```

This query groups records by the specified columns and returns only groups with more than one record.

Example for a customer table where email should be unique:

```
sql

SELECT email, COUNT(*) as count
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

More advanced example with multiple identifiers:

sql

```
SELECT
    first_name,
    last_name,
    date_of_birth,
    COUNT(*) as count
FROM
    customers
GROUP BY
    first_name, last_name, date_of_birth
HAVING
    COUNT(*) > 1;
```

Finding complete duplicate rows:

sql

```
SELECT
    *,
    COUNT(*) OVER(PARTITION BY column1, column2, ...) as dup_count
FROM
    table_name
```

Finding duplicate records with their IDs:

sql

```
WITH duplicates AS (  
    SELECT  
        column1, column2, ...,  
        ROW_NUMBER() OVER(PARTITION BY column1, column2, ... ORDER BY id) as row_id  
    FROM  
        table_name  
)  
SELECT * FROM duplicates WHERE row_id > 1;
```

In production systems, I've implemented automated data quality checks that regularly scan for duplicates based on business rules. For a financial services application, we created a PostgreSQL function that identified potential duplicate transactions using a combination of amount, account, timestamp proximity, and fuzzy matching on descriptions. This approach caught several double-charge issues before they affected customers.

Q: How would you read large sets of data from a CSV and insert into a database efficiently?

A: Handling large CSV imports requires a balanced approach between memory efficiency, performance, and data integrity. Here's my comprehensive strategy:

1. Using native database tools (fastest approach):

sql

```
-- PostgreSQL COPY command
COPY table_name(column1, column2, ...)
FROM '/path/to/file.csv'
WITH CSV HEADER DELIMITER ',' NULL 'NULL';

-- MySQL LOAD DATA
LOAD DATA INFILE '/path/to/file.csv'
INTO TABLE table_name
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

2. Node.js stream-based approach:

javascript

```
const fs = require('fs');
const { Pool } = require('pg');
const csv = require('csv-parser');
const { Transform } = require('stream');

const pool = new Pool(dbConfig);
const client = await pool.connect();

// Begin transaction
await client.query('BEGIN');

try {
  // Prepare batch insert
  const stmt = await client.prepare('insert_stmt',
    'INSERT INTO table_name(col1, col2) VALUES($1, $2)');

  let batchSize = 0;
  const batchLimit = 5000;
  let batchPromises = [];

  fs.createReadStream('large_file.csv')
    .pipe(csv())
    .pipe(new Transform({
      objectMode: true,
      transform: async (row, encoding, callback) => {
        // Add to batch
        batchPromises.push(client.execute('insert_stmt', [row.col1, row.col2]));
        batchSize++;

        // Execute batch when limit reached
        if (batchSize >= batchLimit) {
          this.pause();
        }
      }
    }));
}
```



```

        await Promise.all(batchPromises);
        batchPromises = [];
        batchSize = 0;
        this.resume();
    }
    callback();
}
}))
.on('finish', async () => {
    // Insert remaining records
    if (batchPromises.length > 0) {
        await Promise.all(batchPromises);
    }
    await client.query('COMMIT');
    client.release();
})
.on('error', async (err) => {
    await client.query('ROLLBACK');
    client.release();
    console.error(err);
});
} catch (err) {
    await client.query('ROLLBACK');
    client.release();
    console.error(err);
}
}

```

3. Additional optimization techniques:

- **Pre-validation:** Validate CSV data before attempting inserts
- **Temporary tables:** Load into temp tables first, then insert into production tables

- **Disable indexes:** Drop or disable indexes during bulk loads, then rebuild
- **Transaction management:** Use appropriate transaction sizes to balance commit overhead with rollback scope
- **Connection pooling:** Maintain optimal database connection counts
- **Parallel processing:** Split file and process chunks concurrently (with coordination)

In a recent data migration project, I implemented a hybrid approach where we:

1. Used Node.js streams to read and validate the CSV data
2. Transformed the data into PostgreSQL's binary COPY format
3. Used pg-copy-streams to pipe directly to the database
4. Implemented checkpointing to allow resume after failures

This approach achieved import speeds of over 100,000 records per second while maintaining strong data validation and fault tolerance. The system successfully migrated a 200GB dataset with zero data loss and minimal downtime.

Q: What is database pooling and why is it important?

A: Database connection pooling is a technique that maintains a cache of database connections for reuse, rather than establishing new connections for each client request. It's a critical performance optimization for any production application.

Key aspects of connection pooling:

1. **Connection lifecycle management:** Creates, maintains, and closes connections
2. **Connection reuse:** Allows multiple queries to reuse the same database connection
3. **Request queuing:** Handles scenarios when all connections are in use
4. **Health monitoring:** Validates connections before providing them to application code

Benefits:

- **Improved performance:** Eliminates connection establishment overhead (TCP handshake, authentication)
- **Resource management:** Limits the total number of concurrent database connections
- **Reliability:** Handles connection failures and prevents connection leaks
- **Load balancing:** Can distribute queries across multiple database instances

Implementation in Node.js:

javascript

```
// Using node-postgres
const { Pool } = require('pg');

const pool = new Pool({
  host: 'localhost',
  user: 'postgres',
  password: 'secret',
  database: 'myapp',
  max: 20, // Maximum connections in pool
  idleTimeoutMillis: 30000, // Connection timeout when idle
  connectionTimeoutMillis: 2000 // Connection acquisition timeout
});

async function executeQuery(text, params) {
  const client = await pool.connect();
  try {
    const result = await client.query(text, params);
    return result;
  } finally {
    client.release(); // Return connection to pool
  }
}

// Proper pool shutdown
process.on('SIGTERM', async () => {
  await pool.end();
  process.exit(0);
});
```

Advanced pooling considerations I implement:

- **Pool sizing formula:** $\text{connections} = (\text{core_count} * 2) + \text{effective_spindle_count}$
- **Monitoring:** Track connection utilization, wait time, and errors
- **Connection distribution:** Dedicated pools for different workloads (read vs. write)
- **Circuit breaking:** Automatically remove unhealthy database instances
- **Connection validation:** Test connections before use with lightweight queries

In a high-traffic e-commerce platform I architected, proper connection pooling increased throughput by 300% during peak shopping events while reducing database CPU load by 40%. We also implemented application-level query queuing to prevent database overload during traffic spikes, ensuring stable performance for critical payment processing workflows.

Q: Explain database normalization and when you might choose to denormalize.

A: Database normalization is a systematic approach to organizing database structures to minimize redundancy and dependency, typically through decomposing tables into smaller, well-structured relations.

Normalization Forms:

1. First Normal Form (1NF):

- Each table cell contains a single value
- Each record is unique
- Example: Splitting a multi-value address field into separate street, city, state fields

2. Second Normal Form (2NF):

- Meets all 1NF requirements
- All non-key attributes fully depend on the primary key

- Example: Moving columns that depend only on part of a composite key to a separate table

3. Third Normal Form (3NF):

- Meets all 2NF requirements
- No transitive dependencies (non-key attributes depend only on the key)
- Example: Moving columns that depend on other non-key columns to separate tables

4. BCNF (Boyce-Codd Normal Form):

- A stricter version of 3NF
- Every determinant must be a candidate key

5. Fourth and Fifth Normal Forms:

- Handle multi-valued dependencies and join dependencies
- Rarely used in practical applications

Benefits of Normalization:

- Eliminates data redundancy
- Improves data integrity
- Reduces update anomalies
- More flexible database design
- Efficient storage utilization

When to Denormalize:

Denormalization strategically adds redundancy to improve performance. I consider denormalization when:

1. **Read-heavy workloads:** When reads significantly outnumber writes

2. **Reporting and analytics:** For complex aggregations and calculations
3. **Performance bottlenecks:** When joins become prohibitively expensive
4. **Specific query patterns:** When certain access patterns dominate

Denormalization Techniques:

sql

-- Example: Adding a redundant 'total_orders' column to customers

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  total_orders INT -- Denormalized column  
);
```

-- Example: Creating a materialized view for reporting

```
CREATE MATERIALIZED VIEW monthly_sales AS  
SELECT  
  DATE_TRUNC('month', order_date) AS month,  
  product_id,  
  SUM(quantity) AS units_sold,  
  SUM(price * quantity) AS revenue  
FROM orders  
JOIN order_items USING (order_id)  
GROUP BY 1, 2;
```

Real-world example:

In an e-commerce system I architected, we kept a normalized schema for transaction processing but created denormalized data structures for:

1. Product catalog with embedded category hierarchies for faster browsing

2. Order history with denormalized product details for customer profiles
3. Aggregate tables for sales reporting updated through event-driven processes

This hybrid approach maintained data integrity while achieving sub-100ms response times for critical user flows. We implemented a change data capture (CDC) system to propagate updates from the normalized operational database to the denormalized read models.

The key is balance—I recommend thoughtful, targeted denormalization based on actual performance metrics rather than premature optimization.

Q: How would you optimize slow queries? What are best practices for indexing?

A: Query optimization and proper indexing are essential for database performance at scale. Here's my systematic approach:

1. Query Analysis and Optimization:

First, identify slow queries with profiling tools:

```
sql

-- PostgreSQL
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 123;

-- MySQL
EXPLAIN FORMAT=JSON SELECT * FROM orders WHERE customer_id = 123;
```

Common optimizations:

- Rewrite to reduce data scanned (avoid `(SELECT *)`)
- Add necessary WHERE clauses to limit result sets
- Use JOINS correctly (prefer INNER JOINS where possible)

- Consider query rewriting to avoid subqueries when possible
- Leverage window functions instead of self-joins for analytical queries

2. Indexing Strategy:

Basic index types:

- B-tree indexes (default, good for equality and range queries)
- Hash indexes (equality operations only)
- GIN/GiST indexes (full-text search, arrays, JSON)
- Spatial indexes (geographic data)

Index creation examples:

sql

```
-- Basic single-column index
CREATE INDEX idx_customers_email ON customers(email);

-- Composite index (order matters!)
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);

-- Functional index
CREATE INDEX idx_lower_email ON customers(LOWER(email));

-- Partial index
CREATE INDEX idx_active_orders ON orders(order_date)
WHERE status = 'active';

-- Covering index (includes all fields needed by query)
CREATE INDEX idx_products_covering ON products(category_id, created_at)
INCLUDE (name, price);
```

3. Indexing best practices:

- **Index selectivity:** Index columns with high cardinality
- **Index order:** Most selective columns first for composite indexes
- **Index coverage:** Include necessary columns in the index for covering queries
- **Index maintenance:** Regularly rebuild/reorg fragmented indexes
- **Balance:** Too many indexes degrade write performance

4. Additional optimization techniques:

- **Materialized views:** Pre-compute complex queries

- **Partitioning:** Split large tables by date/range/hash
- **Caching:** Implement application-level caching for frequent queries
- **Query hints:** Guide the optimizer when needed (use cautiously)

Real-world example:

At a fintech company where I served as lead architect, we tackled a performance issue with transaction history searches:

1. **Initial analysis:** EXPLAIN showed a sequential scan on a 50M-row table
2. **Initial index approach:** Added a composite index on (user_id, transaction_date)
3. **Further analysis:** Discovered common filtering by transaction type
4. **Revised strategy:**
 - Created partial indexes for each common transaction type
 - Added covering columns to eliminate lookups
 - Implemented table partitioning by date range
 - Added application-level cache for recent transactions

Results:

- Query time reduced from 3-5 seconds to <50ms
- Index storage overhead decreased by 40% compared to a single large index
- Write performance remained good due to focused indexing

The key to success was a data-driven approach—analyzing query patterns through monitoring tools, benchmarking different index strategies, and continuous measurement after implementation.

Q: What is the difference between LEFT JOIN and RIGHT JOIN in SQL?

A: LEFT JOIN and RIGHT JOIN are SQL operations for combining data from multiple tables with different behaviors regarding which records are included in the result set.

LEFT JOIN:

- Returns all records from the left table (first table mentioned)
- Includes matching records from the right table
- For left table records with no match, NULL values are filled for right table columns

RIGHT JOIN:

- Returns all records from the right table (second table mentioned)
- Includes matching records from the left table
- For right table records with no match, NULL values are filled for left table columns

Visual representation:

Left Table	Right Table
A	1
B	2
C	3

LEFT JOIN example:

```
sql
```

```
SELECT
    users.name,
    orders.order_id
FROM
    users
LEFT JOIN
    orders ON users.id = orders.user_id;
```

This returns all users, even those who haven't placed orders (with NULL for order_id).

RIGHT JOIN example:

```
sql
```

```
SELECT
    products.name,
    categories.category_name
FROM
    products
RIGHT JOIN
    categories ON products.category_id = categories.id;
```

This returns all categories, even those without products (with NULL for product name).

Practical considerations:

- Most SQL developers prefer standardizing on LEFT JOINs for readability
- A RIGHT JOIN can always be rewritten as a LEFT JOIN by swapping the table order

Advanced JOIN patterns I use in production:

```
sql

-- Finding records that exist in one table but not another
SELECT
    customers.customer_id,
    customers.name
FROM
    customers
LEFT JOIN
    orders ON customers.customer_id = orders.customer_id
WHERE
    orders.order_id IS NULL;
-- This identifies customers who haven't placed orders

-- Comparing data between environments (dev vs. prod)
SELECT
    dev.id,
    dev.config_value AS dev_value,
    prod.config_value AS prod_value
FROM
    dev.configurations dev
FULL OUTER JOIN
    prod.configurations prod ON dev.id = prod.id
WHERE
    dev.config_value IS NULL
    OR prod.config_value IS NULL
    OR dev.config_value != prod.config_value;
```

In my role leading database architecture for a financial services platform, I implemented a monitoring system that used LEFT JOINS to identify data discrepancies between customer

records and transaction histories. This approach helped us identify and reconcile over 200 orphaned transactions that would have otherwise caused reporting inconsistencies.

Q: How would you write a SQL query to find the second highest salary?

A: Finding the second highest salary is a classic SQL interview question that tests understanding of ranking functions, subqueries, and data sorting. Here are multiple approaches to solve this problem:

Approach 1: Using ORDER BY with LIMIT/OFFSET:

sql

```
SELECT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Approach 2: Using subquery with MAX:

sql

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Approach 3: Using window functions (most flexible):

sql

```
SELECT salary
FROM (
    SELECT
        salary,
        DENSE_RANK() OVER (ORDER BY salary DESC) as rank
    FROM employees
) ranked
WHERE rank = 2;
```

Why I prefer the window function approach:

1. It handles ties correctly (if multiple employees have the second highest salary)
2. It's easily modifiable to find any Nth highest salary
3. It works consistently across different database systems
4. It can be extended to include additional employee information

Real-world application:

In salary benchmarking reports I developed for HR analytics, we extended this pattern to create percentile-based compensation bands:

sql

```
SELECT
    department,
    job_title,
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1_salary,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary,
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3_salary
FROM
    employees
GROUP BY
    department, job_title
HAVING
    COUNT(*) >= 5;
```

This more sophisticated approach provided valuable insights for compensation reviews and helped identify departments with salary inequities.

ORM Technology

Q: What are the key differences between TypeORM and Sequelize?

A: TypeORM and Sequelize are both popular Object-Relational Mapping (ORM) libraries for Node.js, but they have distinct characteristics that make each more suitable for different projects.

Core Differences:

Feature	TypeORM	Sequelize
Primary Language	TypeScript-first	JavaScript-first with TypeScript support
Schema Definition	Decorator/annotation-based	Model definition objects
Database Support	Wide range (PostgreSQL, MySQL, SQLite, MongoDB, etc.)	Wide range but stronger PostgreSQL/MySQL focus
Query Building	Query builder + repositories	Model methods + Sequelize.query
Migrations	Built-in CLI	Sequelize CLI (separate package)
Relations	Extensive support (eager/lazy loading)	Good support but sometimes requires manual handling
TypeScript Integration	Native, first-class	Added as a layer on top

TypeORM Example:

typescript

// Entity definition

@Entity()

export class User {

@PrimaryGeneratedColumn()

id: number;

@Column({ length: 100 })

name: string;

@OneToMany(() => Post, post => post.author)

posts: Post[];

}

// Repository usage

const userRepository = dataSource.getRepository(User);

const users = await userRepository.find({

where: { name: Like('%John%') },

relations: ['posts']

});

Sequelize Example:

typescript

```
// Model definition
const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  name: {
    type: DataTypes.STRING(100)
  }
});

User.hasMany(Post);

// Query
const users = await User.findAll({
  where: {
    name: { [Op.like]: '%John%' }
  },
  include: [Post]
});
```

When I choose TypeORM:

- For TypeScript projects where type safety is critical
- When the domain model is complex with intricate relationships
- For projects that need powerful query building capabilities
- When the project benefits from decorator-based metadata

When I choose Sequelize:

- For JavaScript-heavy projects or teams more familiar with JS
- When migrating from an existing Sequelize codebase
- When transaction management is particularly important
- For projects requiring well-established patterns with extensive community examples

In a recent microservices architecture I designed, we used TypeORM for our core domain services (where the complex entity relationships benefited from TypeORM's strong typing) and Sequelize for simpler, data-access-focused services. This hybrid approach leveraged the strengths of each ORM for different use cases within the same ecosystem.

API Security and Scalability

Q: What are the best practices for securing APIs?

A: As a senior engineer who has built and secured numerous production APIs, I approach API security through multiple complementary layers:

1. Authentication & Authorization

- **JWT-based authentication** with proper signing algorithms (RS256 preferred over HS256)
- **Short token expiration** with refresh token rotation
- **OAuth 2.0/OpenID Connect** for federated authentication
- **Role-Based Access Control (RBAC)** or **Attribute-Based Access Control (ABAC)**
- **API keys** for service-to-service communication with regular rotation

Implementation example:

javascript

```
// JWT verification middleware
const verifyToken = async (req, res, next) => {
  try {
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) return res.status(401).json({ message: 'No token provided' });

    const decoded = jwt.verify(token, PUBLIC_KEY, { algorithms: ['RS256'] });

    req.user = decoded;

    // Verify token hasn't been revoked
    const isRevoked = await tokenService.isRevoked(token);
    if (isRevoked) return res.status(401).json({ message: 'Token revoked' });

    next();
  } catch (error) {
    if (error instanceof jwt.TokenExpiredError) {
      return res.status(401).json({ message: 'Token expired' });
    }
    return res.status(401).json({ message: 'Invalid token' });
  }
};
```

2. Transport Security

- **HTTPS only** (TLS 1.3 preferred)
- **HTTP Strict Transport Security (HSTS)** headers
- Proper certificate management and renewal
- **Certificate pinning** for mobile clients

3. Input Validation & Output Encoding

- **Schema validation** for all inputs (using libraries like Joi, Zod, or class-validator)
- Parameterized queries to prevent SQL injection
- Contextual output encoding to prevent XSS
- Content Security Policy headers

Example with schema validation:

typescript

```
import { z } from 'zod';

const userSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
  role: z.enum(['user', 'admin']).default('user')
});

app.post('/api/users', async (req, res) => {
  try {
    const validatedData = userSchema.parse(req.body);
    // Proceed with validated data
  } catch (error) {
    return res.status(400).json({ errors: error.errors });
  }
});
```

4. Rate Limiting & Throttling

- **IP-based rate limiting** for public endpoints

- **User-based rate limiting** for authenticated requests
- **Graduated throttling** that increases delays with repeated failures
- **Concurrent request limiting** to prevent resource exhaustion

5. API-Specific Security Controls

- **CORS configuration** with appropriate origins
- **Request payload size limits**
- **JSON Web Encryption (JWE)** for sensitive data
- **API versioning** for secure evolution

6. Monitoring & Threat Detection

- **Real-time monitoring** for unusual patterns
- **Automated alerts** for suspicious activity
- **Audit logging** of all security events
- Integration with **SIEM** solutions

Real-world implementation:

For a financial services API I architected, we implemented a defense-in-depth approach:

1. Multi-factor authentication for sensitive operations
2. Continuous token validation against a centralized token store
3. Request signing with HMAC for non-repudiation
4. AI-powered anomaly detection for transaction patterns
5. Geo-velocity checks to identify impossible travel scenarios
6. Regular penetration testing and bug bounty program

These measures successfully prevented numerous attack attempts while maintaining excellent performance characteristics under load.

Q: How would you make a server scalable without using cloud services?

A: Creating a scalable server architecture without cloud services requires careful planning of both hardware and software components. Here's my approach as someone who has built high-availability systems in both cloud and on-premise environments:

1. Hardware Infrastructure

- **Load balancing:** Deploy hardware load balancers (F5, Citrix NetScaler) or software alternatives (HAProxy, NGINX)
- **Server redundancy:** Implement N+1 or N+2 server configurations
- **Network segmentation:** Separate traffic types with VLANs and multiple NICs
- **Storage solutions:** Use redundant SAN/NAS with proper RAID configurations
- **Multiple data centers:** Geographically distributed locations for disaster recovery

2. Software Architecture

- **Stateless design:** Ensure application components maintain no local state
- **Distributed caching:** Implement Redis or Memcached clusters
- **Message queues:** Use RabbitMQ or Kafka for asynchronous processing
- **Database scaling:** Implement master-slave replication, sharding, and read replicas
- **Service-oriented architecture:** Break monolithic applications into smaller services

Load Balancing Configuration Example:

nginx

NGINX Load Balancer

```
http {  
    upstream backend {  
        least_conn; # Connection distribution method  
        server backend1.example.com max_fails=3 fail_timeout=30s;  
        server backend2.example.com max_fails=3 fail_timeout=30s;  
        server backend3.example.com max_fails=3 fail_timeout=30s backup;  
    }  
  
    server {  
        listen 80;  
        location / {  
            proxy_pass http://backend;  
            proxy_set_header Host $host;  
            proxy_set_header X-Real-IP $remote_addr;  
            proxy_next_upstream error timeout http_500;  
        }  
    }  
}
```

3. Scaling Strategies

- **Horizontal scaling:** Add more servers to distribute load
- **Vertical scaling:** Upgrade existing server resources (CPU, RAM)
- **Database read/write splitting:** Direct writes to master, reads to replicas
- **Caching tiers:** Implement browser, CDN, application, and database caching
- **Asynchronous processing:** Convert synchronous operations to background jobs

4. Performance Optimization

- **Connection pooling:** Optimize database and external service connections
- **Response compression:** Implement GZIP/Brotli for network traffic
- **Static asset optimization:** Image optimization, minification, bundling
- **Efficient queries:** Optimize database access patterns and indexes
- **Resource monitoring:** Implement detailed metrics collection

5. High Availability Configuration

- **Service discovery:** Implement Consul or etcd for service registration
- **Automated failover:** Configure automatic redirection when services fail
- **Health checks:** Regular status monitoring with automatic remediation
- **Rolling deployments:** Zero-downtime deployment processes

Real-world implementation:

For a payment processing system I designed for a financial institution with strict data residency requirements:

1. We implemented a three-tier architecture with web, application, and database servers
2. Used HAProxy for layer 7 load balancing with sticky sessions based on hashed client IDs
3. Deployed MariaDB with Galera Cluster for multi-master replication
4. Implemented data sharding based on customer geographic regions
5. Used Redis clusters for distributed locking and caching
6. Built a custom service registry with health monitoring
7. Implemented circuit breakers to prevent cascading failures

This architecture successfully handled 3000+ transactions per second with 99.99% uptime over a two-year period, all without cloud dependencies.

Q: How would you implement logging and monitoring for error detection?

A: Effective logging and monitoring are critical for maintaining reliable production systems. Here's my comprehensive approach:

1. Structured Logging Strategy

- **Log levels:** Clear delineation between DEBUG, INFO, WARN, ERROR, FATAL
- **Structured format:** JSON-based logging for machine readability
- **Contextual information:** Include request IDs, user IDs, service names
- **PII handling:** Mask sensitive data like emails, tokens, passwords
- **Sample rate control:** Adjust logging verbosity based on traffic patterns

Implementation example:

typescript

```
import winston from 'winston';
import { v4 as uuidv4 } from 'uuid';

// Create request context middleware
app.use((req, res, next) => {
  req.context = {
    requestId: req.headers['x-request-id'] || uuidv4(),
    userId: req.user?.id || 'anonymous',
    ip: req.ip,
    userAgent: req.headers['user-agent']
  };
  next();
});

// Configure structured logger
const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  defaultMeta: { service: 'api-service' },
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'errors.log', level: 'error' })
  ]
});

// Error handling middleware
app.use((err, req, res, next) => {
  const errorId = uuidv4();
```

```
logger.error({
  message: err.message,
  stack: err.stack,
  errorId,
  context: req.context,
  path: req.path,
  method: req.method,
  query: req.query,
  // Avoid logging entire body for privacy/size reasons
  bodyKeys: Object.keys(req.body || {})
});

res.status(err.status || 500).json({
  error: process.env.NODE_ENV === 'production'
    ? 'An unexpected error occurred'
    : err.message,
  errorId // For correlation with logs
});
});
```

2. Monitoring Infrastructure

- **Application metrics:** Response times, error rates, request counts
- **System metrics:** CPU, memory, disk, network utilization
- **Business metrics:** Transaction rates, user activity, conversion rates
- **Dependencies:** Database response times, third-party API status
- **Custom events:** Feature usage, business process completions

3. Alerting System

- **Multi-level alerts:** Warning, critical, emergency thresholds

- **Alert routing:** Different channels based on severity
- **Aggregation logic:** Prevent alert storms with intelligent grouping
- **On-call rotation:** Automated escalation policies
- **Playbooks:** Clear remediation steps for common issues

4. Visualization and Analysis

- **Real-time dashboards:** Service status and key metrics
- **Trend analysis:** Identifying slow degradations over time
- **Correlation tools:** Connecting related events across services
- **Anomaly detection:** ML-based pattern recognition for unusual behavior

5. Advanced Error Detection

- **Distributed tracing:** End-to-end request visualization (Jaeger, Zipkin)
- **Error categorization:** Group similar errors for prioritization
- **Root cause analysis:** Automated identification of error sources
- **Proactive testing:** Chaos engineering to identify weak points
- **User impact assessment:** Correlate errors with user experience

Real-world implementation:

At a fintech company where I led the platform engineering team, we built an observability system that included:

1. Structured logging with contextual request tracing across 20+ microservices
2. Prometheus for metrics collection with Grafana dashboards for visualization
3. Elasticsearch, Logstash, and Kibana (ELK) for log aggregation and analysis
4. Custom anomaly detection using statistical models to identify unusual error patterns

5. PagerDuty integration with intelligent alert routing based on service ownership
6. Automated incident response system that gathered relevant logs, metrics, and recent deployments during incidents

This system reduced our mean time to detection (MTTD) for critical issues from hours to minutes and enabled us to proactively address 70% of potential incidents before they impacted users.

Q: What are the most important HTTP status codes and when should they be used?

A: HTTP status codes are crucial for communicating the result of API requests. As a senior engineer who's designed and implemented numerous REST APIs, I prioritize clear, consistent status code usage to enhance API usability and maintain proper semantics.

Key Status Codes by Category:

2xx - Success

- **200 OK:** Standard success response for most GET requests and successful operations
- **201 Created:** Resource successfully created (POST/PUT that creates new resources)
- **202 Accepted:** Request accepted for processing but not yet completed (for async operations)
- **204 No Content:** Successful operation with no response body (often used for DELETE)

3xx - Redirection

- **301 Moved Permanently:** Resource has a new permanent URI (update bookmarks/references)
- **302 Found:** Temporary redirect
- **304 Not Modified:** Used with conditional requests (If-Modified-Since) to reduce bandwidth

4xx - Client Errors

- **400 Bad Request:** Malformed request syntax or invalid request message
- **401 Unauthorized:** Authentication required or failed (more accurately "Unauthenticated")
- **403 Forbidden:** Server understood but refuses to authorize (permissions issue)
- **404 Not Found:** Resource doesn't exist
- **409 Conflict:** Request conflicts with current state (e.g., duplicate resource, version conflict)
- **422 Unprocessable Entity:** Semantically incorrect request (validation errors)
- **429 Too Many Requests:** Rate limiting applied

5xx - Server Errors

- **500 Internal Server Error:** Generic server-side error
- **502 Bad Gateway:** Invalid response from upstream server
- **503 Service Unavailable:** Server temporarily unavailable (maintenance, overload)
- **504 Gateway Timeout:** Upstream server did not respond in time

Implementation examples:

typescript

```
// RESTful controller with appropriate status codes
class UsersController {
  // GET /users/:id
  async getUser(req, res) {
    try {
      const userId = req.params.id;
      const user = await userService.findById(userId);

      if (!user) {
        return res.status(404).json({
          error: 'User not found'
        });
      }

      return res.status(200).json(user);
    } catch (error) {
      logger.error('Error fetching user', { error, userId: req.params.id });
      return res.status(500).json({
        error: 'Internal server error',
        requestId: req.requestId
      });
    }
  }
}

// POST /users
async createUser(req, res) {
  try {
    // Validation
    const validation = userSchema.validate(req.body);
    if (validation.error) {
      return res.status(422).json({
        error: 'Validation failed',
      });
    }
  }
}
```

```

        details: validation.error.details
    });
}

// Check for existing user
const existingUser = await userService.findByEmail(req.body.email);
if (existingUser) {
    return res.status(409).json({
        error: 'User with this email already exists'
    });
}

// Create user
const newUser = await userService.create(req.body);

// Return created resource with location header
res.setHeader('Location', `/api/users/${newUser.id}`);
return res.status(201).json(newUser);
} catch (error) {
    logger.error('Error creating user', { error });
    return res.status(500).json({
        error: 'Internal server error',
        requestId: req.requestId
    });
}
}

// DELETE /users/:id
async deleteUser(req, res) {
    try {
        const userId = req.params.id;
        const deleted = await userService.delete(userId);
    }
}

```

```
    if (!deleted) {
      return res.status(404).json({
        error: 'User not found'
      });
    }

    return res.status(204).end();
  } catch (error) {
    logger.error('Error deleting user', { error, userId: req.params.id });
    return res.status(500).json({
      error: 'Internal server error',
      requestId: req.requestId
    });
  }
}
```

Good practices I implement:

1. Return appropriate headers with status codes (e.g., Location with 201)
2. Include helpful error messages and validation details
3. Use consistent error formats across the API
4. Document all possible status codes in API specifications
5. Add request identifiers to link client errors with server logs

In a recent API gateway project, we implemented standardized error handling that transformed various backend error types into consistent HTTP status codes. This approach improved client integration experience and helped developers quickly identify and fix issues based on the specific status codes returned.

NestJS and Express.js

Q: What are interceptors and guards in NestJS?

A: In NestJS, interceptors and guards are powerful features that enable aspect-oriented programming and declarative security for your application. They're part of what makes NestJS so powerful for enterprise applications.

Guards:

Guards determine whether a request should be handled by the route handler based on certain conditions (permissions, roles, validation). They run **before** any middleware or interceptor.

Key characteristics of guards:

- Implement `CanActivate` interface
- Return boolean or throw exceptions
- Execute before route handlers
- Ideal for authentication and authorization
- Have access to `ExecutionContext` (request details)

Example guard implementation:

typescript


```
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    // Get required roles from handler metadata
    const requiredRoles = this.reflector.get<string[]>(
      'roles',
      context.getHandler()
    );

    if (!requiredRoles) {
      return true; // No roles required
    }

    const request = context.switchToHttp().getRequest();
    const user = request.user;

    if (!user) {
      return false; // No user attached to request
    }

    // Check if user has required roles
    return requiredRoles.some(role => user.roles.includes(role));
  }
}

// Usage with decorator
@Roles('admin')
@UseGuards(RolesGuard)
@Get('admin-dashboard')
getAdminDashboard() {
```

```
// Only accessible to admins  
}
```

Interceptors:

Interceptors provide a mechanism to:

- Bind extra logic before/after method execution
- Transform the result returned from a function
- Transform exceptions thrown from a function
- Extend basic function behavior
- Override a function depending on specific conditions

Key characteristics of interceptors:

- Implement `NestInterceptor` interface
- Leverage RxJS observables for powerful transformations
- Can manipulate both request and response
- Perfect for cross-cutting concerns like logging, caching, and timing

Example interceptor implementation:

typescript

```

@Inject()
export class LoggingInterceptor implements NestInterceptor {
  constructor(private logger: LoggerService) {}

  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const request = context.switchToHttp().getRequest();
    const method = request.method;
    const url = request.url;
    const userAgent = request.headers['user-agent'] || '';
    const userId = request.user?.id || 'anonymous';

    const now = Date.now();

    this.logger.log(`Request: ${method} ${url} - User: ${userId} - UserAgent: ${u

    return next.handle().pipe(
      tap({
        next: (data) => {
          const response = context.switchToHttp().getResponse();
          const delay = Date.now() - now;

          this.logger.log(
            `Response: ${method} ${url} - ${response.statusCode} - ${delay}ms`
          );
        },
        error: (error) => {
          const delay = Date.now() - now;
          this.logger.error(
            `Error: ${method} ${url} - ${error.status || 500} - ${delay}ms`,
            error.stack
          );
        }
      })
    );
  }
}

```

```

    })
  );
}

// Global usage
@Module({
  providers: [
    {
      provide: APP_INTERCEPTOR,
      useClass: LoggingInterceptor
    }
  ]
})
export class AppModule {}

```

Advanced use cases I've implemented:

1. **Caching interceptors** that stored responses in Redis with TTL based on route metadata
2. **Rate limiting interceptors** that tracked API usage per client
3. **Response transformation interceptors** that standardized API responses
4. **Metrics interceptors** that reported request durations to monitoring systems
5. **Authorization guards** that integrated with RBAC systems for fine-grained permissions

Both guards and interceptors make NestJS particularly well-suited for enterprise applications where cross-cutting concerns and separation of responsibilities are critical.

Q: Explain dependency injection in NestJS. How does IoC work?

A: Dependency Injection (DI) and Inversion of Control (IoC) are fundamental concepts in NestJS that enable building testable, maintainable, and loosely coupled applications.

Core Concepts:

Dependency Injection (DI) is a design pattern where:

- Components receive their dependencies rather than creating them
- Classes declare dependencies through constructor parameters
- The framework (NestJS) handles instantiation and lifecycle

Inversion of Control (IoC) refers to:

- Control over object creation and lifecycle being "inverted" from application code to the framework
- The application doesn't call the framework; the framework calls your code
- Classes focus on functionality, not dependency management

How NestJS implements DI and IoC:

1. **Provider Registration:** Services are registered as providers in modules
2. **Dependency Declaration:** Classes declare their dependencies in constructors
3. **Dependency Resolution:** NestJS resolves dependencies and creates the dependency graph
4. **Instance Management:** NestJS maintains instances based on configured scope

Basic Example:

typescript

```
// Service definition
@Injectable()
export class UserService {
  constructor(
    private readonly userRepository: UserRepository,
    private readonly mailService: MailService
  ) {}

  async createUser(userData: CreateUserDto): Promise<User> {
    const user = await this.userRepository.create(userData);
    await this.mailService.sendWelcomeEmail(user.email);
    return user;
  }
}
```

```
// Controller using the service
@Controller('users')
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Post()
  createUser(@Body() createUserDto: CreateUserDto) {
    return this.userService.createUser(createUserDto);
  }
}
```

```
// Module registration
@Module({
  controllers: [UserController],
  providers: [UserService, UserRepository, MailService]
```



```
})  
export class UserModule {}
```

Advanced DI features:

1. **Custom Providers:** Various ways to provide dependencies

typescript

```
@Module({
  providers: [
    // Standard provider
    UserService,

    // Value provider
    {
      provide: 'API_KEY',
      useValue: process.env.API_KEY
    },

    // Class provider
    {
      provide: PaymentService,
      useClass:
        process.env.PAYMENT_PROVIDER === 'stripe'
          ? StripePaymentService
          : PayPalPaymentService
    },

    // Factory provider
    {
      provide: 'DATABASE_CONNECTION',
      useFactory: async (configService: ConfigService) => {
        const dbConfig = configService.get('database');
        const connection = await createConnection(dbConfig);
        return connection;
      },
      inject: [ConfigService] // Dependencies of the factory
    }
  ]
})
```

```
})  
export class AppModule {}
```

2. **Scopes:** Control instance lifecycle

typescript

```
@Injectable({ scope: Scope.REQUEST })  
export class RequestScopedService {  
  // New instance created for each request  
}
```

```
@Injectable({ scope: Scope.TRANSIENT })  
export class TransientService {  
  // New instance created each time it's injected  
}
```

```
@Injectable() // Default: Scope.DEFAULT (singleton)  
export class SingletonService {  
  // One instance shared across the application  
}
```

3. **Circular Dependencies:** Handled with forwardRef()

typescript

```
@Injectable()
export class ServiceA {
  constructor(
    @Inject(forwardRef(() => ServiceB))
    private serviceB: ServiceB
  ) {}
}
```

Benefits in production applications:

1. **Testability:** Easy mocking of dependencies for unit tests
2. **Maintainability:** Clear dependency graph and separation of concerns
3. **Configurability:** Runtime selection of implementation details
4. **Reusability:** Services can be injected across multiple modules

In a large-scale financial application I architected, we leveraged NestJS's DI system to create a plug-and-play payment processing module that could swap between different payment providers based on configuration. This approach allowed us to maintain a consistent API internally while supporting multiple payment gateways without code changes.

Q: Compare NestJS and Express.js. What does NestJS use under the hood?

A: NestJS and Express.js represent different philosophies for building Node.js applications, with NestJS providing a higher-level framework built on top of platforms like Express.js.

Key Comparisons:

Aspect	Express.js	NestJS
Architecture	Minimal, unopinionated framework	Opinionated, full-featured framework with architectural patterns
Paradigm	Function-based middleware	Object-oriented with decorators
Structure	Provides minimal structure, developers create their own	Enforces modular architecture with clear patterns
TypeScript	Requires setup and configuration	First-class TypeScript support out of the box
Dependency Injection	Not built-in, requires additional libraries	Comprehensive DI system built-in
Testing	Minimal testing utilities	Extensive testing utilities built-in
Ecosystem	Vast ecosystem of middleware	Integrated modules for common functionality
Learning Curve	Lower, fewer concepts	Steeper, more architectural concepts
Enterprise Readiness	Requires additional setup	Enterprise patterns built-in

NestJS Under the Hood:

NestJS is built on top of either Express.js (default) or Fastify as the HTTP server framework. It enhances these platforms with:

1. **Dependency Injection Container:** Provides IoC capabilities
2. **Module System:** Organizes code into cohesive modules
3. **Decorator-based Metadata:** Uses TypeScript decorators for routing, validation, etc.
4. **Interceptors/Filters/Guards/Pipes:** Middleware-like components with specific responsibilities
5. **WebSockets and Microservices Support:** Beyond HTTP capabilities

How NestJS uses Express:

typescript

```
// Express middleware can be used within NestJS
import { Injectable, NestMiddleware } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    console.log(`Request: ${req.method} ${req.url}`);
    next();
  }
}

// Access the underlying Express instance
const app = await NestFactory.create(AppModule);
const expressApp = app.getHttpAdapter().getInstance();
expressApp.use(express.static('public')); // Direct Express middleware
```

Code comparison - Simple API:

Express.js:

javascript

```
const
```